

02. 規格驅動開發 - SDD (Spec-Driven Development)

一、提示詞 (Prompt)

AI 生成結果都是根據提供的 Prompt (提示詞) 作為初始上下文 (Initial Context) 所生成的內容。因此，Prompt 的品質將直接影響 AI 的輸出品質，這也是資訊領域常提到的概念：

Garbage In, Garbage Out (垃圾進，垃圾出)

如果輸入的指令模糊、不完整或方向錯誤，AI 即使擁有強大的能力，也可能產生不理想的結果；相反地，當指令清楚、具體且目標明確時，AI 才能提供更精準、更符合需求的內容。

可以把 AI 想像成一位能力優秀、執行力極高的工讀生，它能快速完成工作，但最終成果的品質，仍然高度取決於你如何下達任務與提供資訊。

好的 Prompt，往往比強大的工具更重要。



二、什麼是 Spec-Driven Development(規格驅動開發)？

AI 就像一位能力超強的工讀生：

- 看過大量程式碼與文件，知識非常豐富。

- 執行速度極快，幾秒鐘就能完成工作。
 - 但他不認識你、不知道你的習慣，也沒有任何默契。
- 因此，如果指令不夠明確，AI 很容易依照自己的理解做出結果，而不是你真正想要的結果。

- **沒有規格時：AI 只能靠猜測**

想像你對工讀生說：

「幫我買一份午餐。」

他可能會：

- 買一份 3000 元的高級法式料理。
- 買一份他認為最健康，但你最討厭的苦瓜便當。
- 買一份份量太少或太多的餐點。

問題不在於工讀生能力不好，而是因為需求不夠明確。

同樣地，當我們直接要求 AI：

「幫我寫一個網站」

AI 只能根據自己的推測產生內容，因此每次對話都可能得到不同結果。

- **SDD 就是給 AI 一份詳細工作清單**

與其說：

「幫我買午餐」

不如提供一份明確規格：

- 預算：120 元以內
- 不要青椒
- 需要附飲料
- 希望能在 15 分鐘內買回來

有了這份規格，任何人都能按照同樣標準完成任務。

在軟體開發中，這份「工作清單」就是 **Specification (規格文件)**。

- **SDD 的核心流程**

- 1. 先寫規格 (Specification)**

先定義需求：

- 功能要做什麼？
- 使用者如何操作？
- 畫面長什麼樣子？
- 資料如何處理？
- 哪些情況需要特別處理？

- 2. 再寫程式碼 (Implementation)**

根據規格進行開發：

- 每一段程式碼都能對應到規格內容。

- 不憑感覺新增功能。
- 不依個人喜好隨意修改。

3. 修改需求時，先改規格

正確流程：

更新規格
↓
更新程式碼
↓
測試驗證

而不是：

直接改程式
↓
想到什麼改什麼
↓
最後不知道改過什麼

• 不使用 SDD 常見問題

▪ 結果不一致

同樣一句話：

幫我做一個會員系統

每次詢問 AI 都可能得到不同版本。

▪ 修改失控

今天加一個功能：

加入會員等級

明天又改：

加入點數系統

後天再改：

加入優惠券

久而久之，連開發者自己都不知道系統為什麼變成這樣。

▪ 無法追蹤需求來源

當主管問：

為什麼這個功能要這樣設計？

如果沒有規格文件，往往只能回答：

當初好像覺得這樣比較方便.....

- 使用 SDD 的好處

- 提升程式碼品質

- 所有功能都有明確依據：

- ◆ 結構更清楚
 - ◆ 維護更容易
 - ◆ 錯誤更容易發現

- 結果可預測

- 無論：

- ◆ 換一位工程師
 - ◆ 換一個 AI 工具
 - ◆ 隔半年重新開發

- 只要依照同一份規格，結果都會高度一致。

- 每次修改都有依據

- 修改不是憑感覺，而是因為：

- ◆ 規格改變了
 - ◆ 需求改變了
 - ◆ 使用者回饋改變了

- 因此每一次調整都有明確原因。

- 容易回溯與維護

- 當未來需要修改系統時，可以快速找到：

- ◆ 原始需求是什麼？
 - ◆ 為什麼要這樣設計？
 - ◆ 哪次需求變更造成目前結果？

- SDD 的真正價值

- SDD 不只是讓 AI 更好用。

- 它的核心目標是：

- 把「猜測」轉換成「明確需求」，把「靈感式開發」轉換成「可追蹤的工程流程」。

- 當規格越清楚：

- AI 產出的結果越穩定
 - 團隊溝通成本越低
 - 專案品質越容易控制

- 這也是專業軟體開發團隊長期採用的工作方式。

- 一句話總結

- 不要直接要求 AI 寫程式，而是先定義規格，再讓 AI 依照規格開發。

規格決定品質，規格越清楚，結果越符合預期。

• 有無 SDD 的「超級工讀生」戰力對比

比較項目	只有模糊需求	擁有 SDD (規格驅動)
需求理解	AI 靠猜測理解需求	AI 依規格精準執行
成果一致性	每次產出都可能不同	結果穩定且可預測
修改次數	反覆來回修正	大幅降低重工
開發效率	經常停下來確認需求	直接朝目標前進
程式品質	容易出現邏輯不一致	架構清晰、品質穩定
文件追蹤	不知道為何這樣設計	所有決策都有依據
團隊協作	容易產生認知落差	所有人依同一份規格工作
AI 協作效果	像在玩猜謎遊戲	像在執行任務清單
專案風險	功能容易失控擴張	需求範圍清楚可控
開發者心情	一直修改、懷疑人生	輕鬆掌握進度與成果

為什麼要使用 SDD?

就像給工讀生一張詳細的工作清單



- 博學多聞：他看過全世界的程式碼，精通各國語言。
- 手速極快：你交代的工作，他幾秒鐘就能交件。
- 致命缺點：他對你完全不瞭解，也沒有任何「常識」或「默契」。

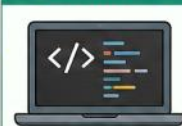
SDD 的核心流程

1 先寫規格 (Spec)



明確定義需求、功能、流程、介面與規則。

2 再寫程式碼



每一段程式碼都必須對應到規格內容。

3 修改流程



更新規格 → 更新程式碼 → 測試驗證。

★ 所有程式碼都必須對應到一份規格。

不用 SDD 的問題



每次跟 AI 對話都產出不同結果

同樣的需求，每次得到的結果都可能不一樣。



改東改西，最後連自己都 不知道做了什麼

功能越改越多，系統越來越亂，難以維護。



無法追蹤「為什麼要這樣改」

沒有紀錄需求來源與變更原因，無法回溯與追蹤。

使用 SDD 的好處



程式碼品質一致、可預測
按照規格開發，結果穩定、結構清晰、容易維護。



每次修改都有明確的目的和依據
需求變更有紀錄，開發更有方向，減少溝通成本。



可以隨時回頭檢視 「當初的需求是什麼」

所有需求與變更都有紀錄，方便回溯與追蹤。



培養專業的工程思維
從規格思考、結構設計到驗證測試，建立正確的開發流程。

想像你請了一位「超級工讀生」

如果你只說：

「幫我買份午餐」

他可能會...

3000 元
法式料理?

苦瓜便當?

但如果你提供一份詳細的規格 (訂購單)

午餐訂購單

預算: 120 元以內

不要青椒

一定要附飲料

15 分鐘內買回來

這就是 SDD 的價值：減少「猜測」，確保結果符合預期!



這就會
符合你的
期待!



一句話總結

不要直接要求 AI 寫程式，而是**先定義規格**，再讓 AI **依照規格開發**。
規格決定品質，規格越清楚，結果越符合預期。



三、提示詞心法

• 提示詞心法：把 AI 當成專業工讀生

下 Prompt 不一定有固定公式，但可以用一個簡單觀念來理解：

AI 就像一位能力很強的專業工讀生。

他學得快、做得快，也懂很多事情，但他對你的專案背景、目標、限制條件完全不清楚。
所以，每次請 AI 幫忙前，都要把任務交代清楚。

1. 講清楚你要做什麼

這一步就像是幫任務下一個明確標題。

不要只說：「幫我寫一個網頁。」

這樣 AI 會不知道你真正想要的是什麼。

比較好的說法是：

「請幫我製作一個『咖啡拉花課程』的落地頁。」

新手可以記住一個原則：

直接說出你想要的最終成果。

就像你去飲料店，不會只說「給我一杯好喝的」，而是會說：

「我要一杯大杯、去冰、無糖拿鐵。」

Prompt 也是一樣，不要讓 AI 猜你的意思。

2. 盡可能提供上下文

AI 不知道你的專案目前長什麼樣子，也不知道你手上已經有哪些資料。

所以你要盡可能提供：

- **現有程式碼**

例如：把完整的 `index.html` 貼給 AI，讓它知道目前網頁結構與插槽位置。

- **已準備好的文案**

例如：標題、副標、課程內容、按鈕文字、圖片關鍵字。

- **使用情境**

例如：這是招生頁、活動頁、作品集頁，或是教學用範例。

上下文給得越完整，AI 產出的結果就越接近你的需求。

簡單來說：

AI 不會自動知道你想做什麼，你給的資訊越清楚，結果就越準確。

3. 定義產出結果

這是最重要的驗收標準。

你要明確告訴 AI：

最後要交出什麼？格式是什麼？有哪些限制？

可以從以下幾個方向說明：

- **檔案格式**

例如：

「請只給我單一 index.html。」

- **技術限制**

例如：

「不能使用 JavaScript。」

「CSS 請使用 class 選擇器，不要使用 ID 選擇器。」

- **結構要求**

例如：「網頁要包含 Hero 區、About 區、課程特色區，並預留圖片插槽。」

- **交件格式**

例如：

「請直接給我完整程式碼，不要只給片段。」

「請分成 HTML、CSS、JS 三個檔案。」

「請用表格整理結果。」

這就像你交代工讀生：

「報告請用 PDF 格式，三頁以內，包含封面、重點整理與結論。」

如果沒有定義清楚，AI 可能會產出看起來很完整，但實際上不能直接使用的內容。

※重要提醒

AI 可能會出錯。

更麻煩的是，AI 出錯時常常看起來很有自信。
所以使用 AI 時，不只要會下 Prompt，也要學會檢查結果。
可以記住這三個步驟：

講清楚目標 → 提供上下文 → 定義產出結果

Prompt 寫得越清楚，AI 就越能成為你的好幫手。

提示詞心法

把 AI 當成專業工讀生，每次請他做事前，都要交代清楚！

給的資訊越清楚，結果就越精準！

AI 能力很好，但對你的專案完全不知情。你給的資訊越完整，產出就越符合需求。

1 講清楚你要做什麼

明確目標

直接說出你想要的最終成果，不要讓 AI 猜你的意圖。

新手好理解的說法

去餐廳會說「我要大杯去冰拿鐵」，而不是「給我一杯好喝的」。
Prompt 也要直接說清楚！

錯誤範例

幫我寫一個網頁。

不知道要做什么...

正確範例

幫我製作一個「咖啡拉花課程」的落地頁。

目標明確，開始工作！

2 盡可能提供上下文

現有程式碼或文案

AI 絕對不知道你想幹嘛，給越多資訊準確度越高。

現有程式碼

把整份 index.html 給 AI，讓它看見插槽 (Slot) 的位置。

準備好的文案

提供標題、內容文字、按鈕文字、圖片關鍵字等具體資訊。

使用情境

說明這是什麼類型的頁面，例如：招生頁、活動頁、作品集頁。

3 定義產出結果

只要零件還是整份、要切、還是要辣

這是最關鍵的「驗收標準」，決定了 AI 交出來的東西能不能直接使用。

檔案格式

例如：請只給我單一的 index.html。

技術限制

例如：不能使用 JavaScript，CSS 要用類別選擇器。

結構要求

例如：要包含 Hero 區、About 區，並預留 Slot 插槽。

交付格式

例如：請直接給我完整程式碼，不要只給片段。

重要提醒

AI 可能會出錯，而且它出錯的時候往往看起來很有自信。所以不只要會下 Prompt，還要學會檢查結果！

講清楚目標 + 提供上下文 → 定義產出結果 = AI 才能成為你的好幫手！

★ Prompt 寫得越清楚，AI 就越能準確理解你的需求，幫你節省時間、提高效率！

四、如何做好 SDD (Specification Driven Development，規格驅動開發)？

SDD 的核心精神很簡單：

先定義規格 (Spec)，再開始開發 (Code)。

不論是人類工程師還是 AI，都必須依照規格工作，而不是想到什麼做什麼。

常見有三種實踐方式，可以依照專案規模與自身經驗選擇。

• 手法一：完整文件法 (專家藍圖模式)

這就像蓋房子前先完成完整建築藍圖。

在正式開發前，先撰寫詳細規格文件，內容可能包含：

- 功能需求
- 畫面設計

- 操作流程
- 資料結構
- 邏輯規則
- 錯誤處理機制
- 邊界條件

接著再交由工程師或 AI 依照規格實作。

優點

- 規格完整且一致
- 開發結果可預測
- 適合多人協作
- 後續維護容易

挑戰

- 撰寫成本高
- 需要具備良好的需求分析能力
- 初學者容易遺漏重要細節

適合情境

- 大型網站
- 商業系統
- 長期維護專案
- 團隊協作開發

• 手法二：AI 引導對話法（面試顧問模式）

如果你只有一個模糊想法，可以讓 AI 先擔任需求分析師。

例如：

我想做一個咖啡課程網站。

此時不要急著要求產生程式碼，而是請 AI：

扮演資深產品經理與前端工程師，請列出完成此網站還需要確認的所有資訊，並逐項向我提問。

接著 AI 會開始詢問：

- 目標客群是誰？
- 網站目的是什麼？
- 需要哪些頁面？
- 是否需要會員系統？
- 是否需要線上付款？
- 品牌風格為何？
- 是否需要響應式設計（RWD）？

透過問答逐步補齊規格。

優點

- 能快速整理想法

- 補足需求盲點
- 不需要一次寫完整規格

挑戰

- 需要多輪對話
- 規格容易散落在聊天紀錄中

適合情境

- 剛開始規劃專案
- 不知道需求怎麼寫
- 初學者練習需求分析

• 手法三：漸進式產出法（樂高組裝模式）

不要一次完成整個專案，而是拆解成多個小模組逐步完成。

例如製作電商網站：

第一階段

先完成導覽列（Navbar）

規格：

- Logo
- 首頁連結
- 商品連結
- 聯絡我們連結
- 手機版漢堡選單

確認沒問題後再進入下一步。

第二階段

完成商品列表區

規格：

- 商品圖片
- 商品名稱
- 商品價格
- 購買按鈕

第三階段

完成商品詳細頁

第四階段

完成購物車功能

第五階段

完成結帳流程

每次只處理一個小模組。

優點

- 容易驗證成果
- 錯誤容易修正

- 可以隨時調整方向
- 學習壓力較小

挑戰

- 需要維持整體架構一致性
- 需定期回顧已完成規格

適合情境

- AI 協作開發
- 學習專案
- MVP (最小可行產品)
- 個人開發者

※給初學者的建議

如果是剛開始使用 AI 開發：

1. 先使用「AI 引導對話法」整理需求
2. 再使用「漸進式產出法」逐步完成功能
3. 專案規模變大後，再學習「完整文件法」

實務上最常見的流程其實是：

AI 引導對話 → 建立規格 → 漸進式開發 → 持續修正規格

這也是目前許多 AI 輔助開發團隊最常採用的 SDD 工作模式。

記住一句話：

不要直接叫 AI 寫程式，而是先讓 AI 幫你把規格寫清楚。

規格越清楚，產出的品質就越穩定；規格越模糊，AI 就越容易自由發揮。



如何做好 SDD (規格驅動開發)？

核心精神：先定義規格 (Spec)，再開始開發 (Code)。規格越清楚，產出的品質就越穩定！

